



NEXT-GEN FINOPS PLATFORM

Cloud Cost Optimizer & Remediation Engine

An API-First solution built with FastAPI, SQLite, and Streamlit designed to identify, analyze, and instantly decommission orphaned infrastructure artifacts.

The \$44B Cloud Waste Problem

A large, bold, blue '30%' is centered on the page. In the background, there is a faint, light blue silhouette of a person's head and shoulders, facing right.

Estimated Public Cloud Waste

Unused Resources Bleed Capital

Data from the 2026 State of FinOps Reports indicate that organisations waste roughly 30% of their compute budgets on under-utilized systems.

The core issue is lack of programmatic action: teams find the waste manually, but fail to generate the precise code loops required to eliminate it without administrative risk.

Primary Waste Vectors Targeted

Unattached Disk Storage

AWS EBS and Azure Managed Disks continue to accrue charges even after their parent VM is terminated. By isolating disks sitting in "available" or "Unattached" states, our optimizer immediately detects pure overhead waste.

Idle & Stopped VMs

Developers routinely spawn virtual machine workloads and abandon them over weekends or following dev cycles. Flagging VM states identified as "idle" or "stopped" isolates high-value cost reduction targets.

Engine Architecture Core Phases



1. Ingestion

Accepts raw cloud cost inventory records via JSON/CSV payloads, mapping structural parameters into standard multi-cloud entities.



2. Analysis

Applies programmatic assessment rules over the dataset to isolate inactive states, validating financial leakages automatically.



3. Remediation

Translates target asset metadata into precise AWS and Azure CLI commands, allowing immediate and secure execution.

Relational Model & Schema



Database Column	Data Type	Core Function	Cloud Mapping API Source
cloud_provider	VARCHAR	Routes cloud engine profiles	Identifies source configuration (AWS/Azure)
resource_id	VARCHAR (Unique)	Prevents ingestion duplicates	Physical Asset Key (EBS ID, Instance ID)
resource_type	VARCHAR	Identifies asset categories	Limits classifications to volume or vm
estimated_monthly_waste	FLOAT	Aggregates potential run-rate savings	Pricing metrics mapped from billing export

API-First Backend Engine

FastAPI + Pandas Parser

The `/import/inventory/` endpoint ingests multi-cloud structured files, loading them into memory-buffered Pandas DataFrames.

This allows rapid parsing and conditional column validation, mapping diverse key schemas into clean tabular database records instantly.

SQLAlchemy SQLite Database

Our persistence layer uses SQLAlchemy mapping over a highly portable, free-tier SQLite local engine.

By enforcing unique keys and indexing `resource_id` parameters, our database acts as a deterministic state ledger for cloud optimization.

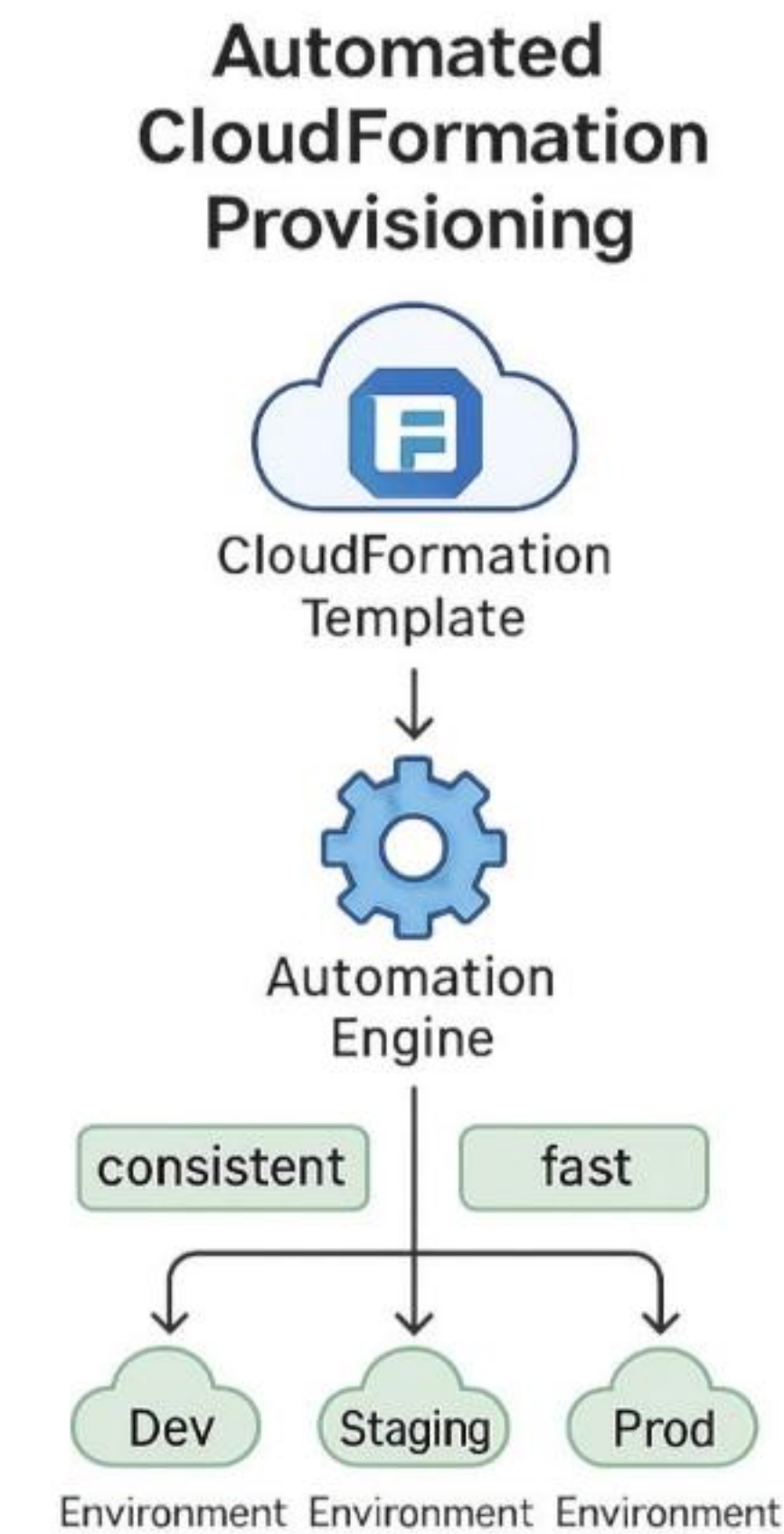
Deterministic CLI Generation

Remediation Translators

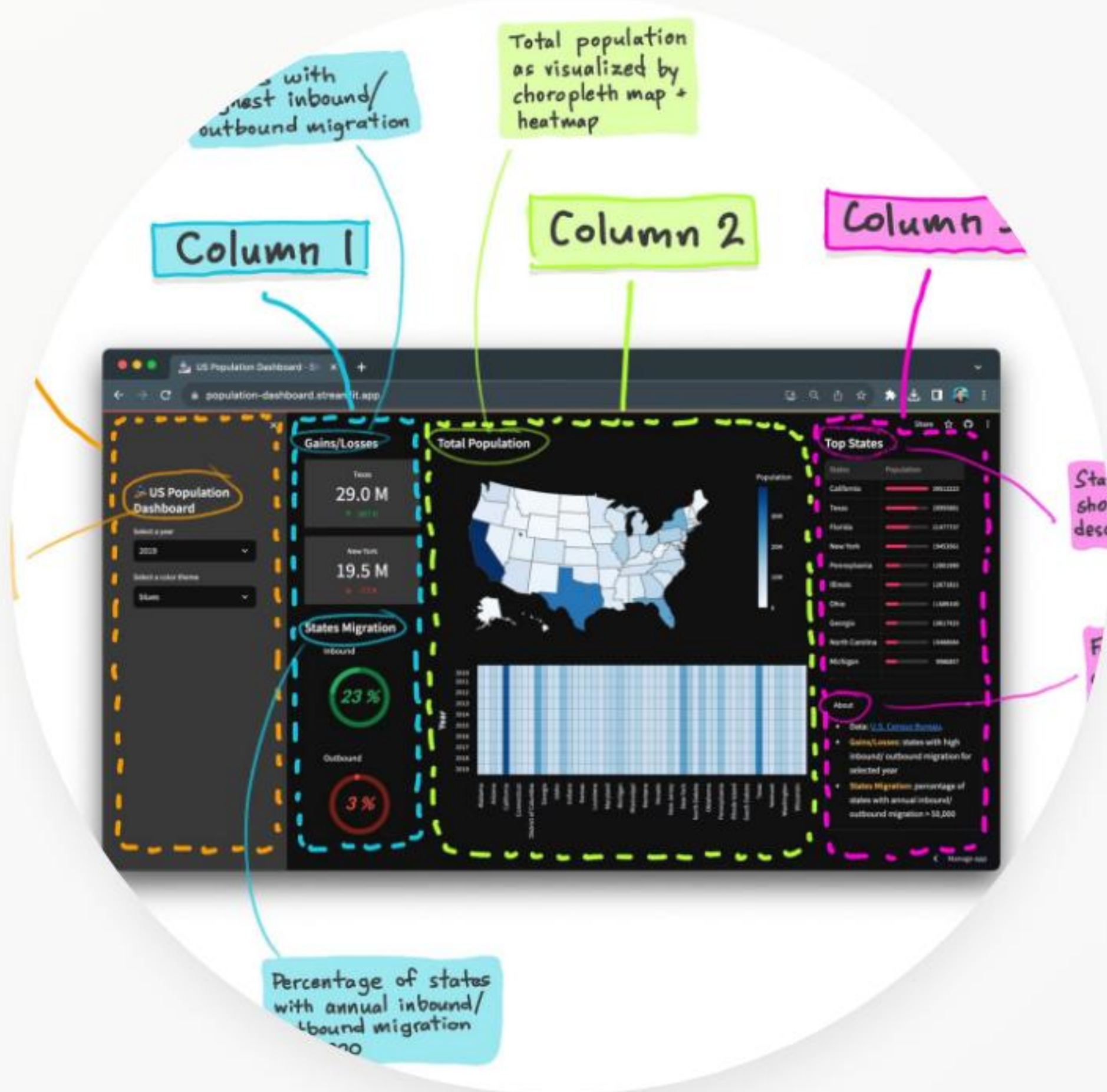
To move from detection to resolution, the backend translates storage and compute identifiers directly into actionable, cloud-native shell commands.

This bypasses complex orchestration engines, generating standard shell files ready to execute safely within a sandbox environment.

Our generator programmatically builds tailored delete-volume, terminate-instances, and Azure resource groups clean-up steps based on active inventory flags.



State Management & Streamlit



WebSocket Vs Serverless State

Our Frontend GUI relies on Streamlit, which leverages continuous WebSocket connectivity to manage state during user interactions.

Deploying Streamlit to serverless frameworks like Vercel fails because containers are recycled constantly, destroying persistent states.

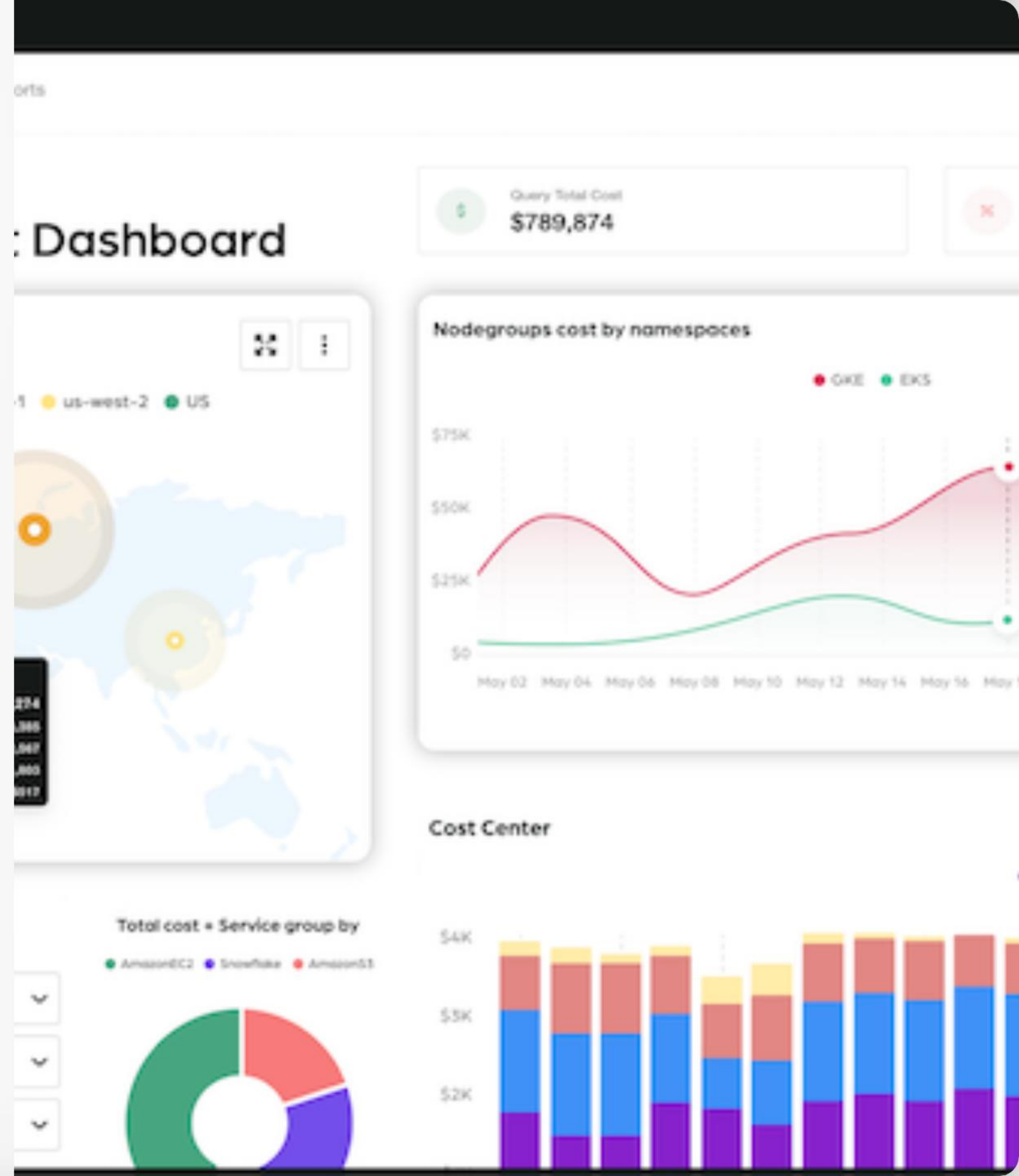
By deploying to Render Web Services instead, our client app retains active WebSocket tunnels and communicates perfectly with the FastAPI endpoint.

Interactive Ingestion Tab

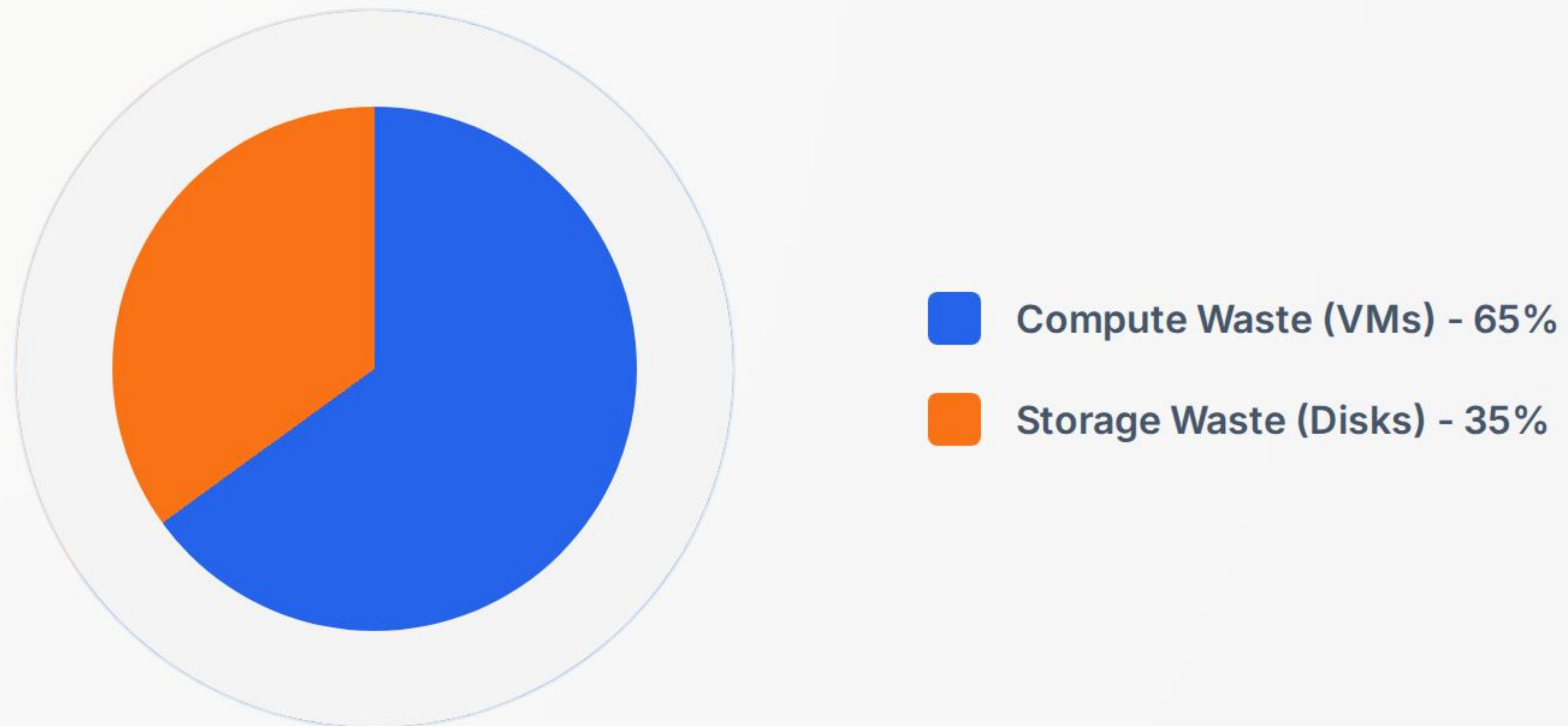
Staging Upload Pipeline

The client app provides a secure file drag-and-drop zone. Users select the cloud target (AWS/Azure) and upload raw infrastructure export files.

This dynamic interface binds the CSV/JSON payload directly to backend API models, triggering instant evaluation runs on-the-fly and highlighting detected anomalies.



Waste Allocation Analysis



Assessments show that while compute resources account for the largest share of dollar waste, orphaned storage volumes represent zero-downtime optimization "quick-wins".

Key Ingestion Schema Rules

-  **ID Primary Constraints:** Enforces uniqueness on all ingested resource IDs to maintain state integrity and avoid duplicate financial reporting.
-  **State Mapping Matchers:** Tracks explicit lifecycle flags such as "available" and "Unattached" for storage, alongside "idle" and "stopped" configurations for VMs.
-  **Savings Prioritizer:** Calculates cost projections, enabling organizations to prioritize clean-up loops based on the highest-value opportunities.

Demo & Deployment Pipeline

Step 2 – UI Deploy

Streamlit client GUI deployed alongside the backend on stateful Render, maintaining active WebSockets.

Step 1 – API Deploy

FastAPI backend engine deployed as a persistent Render Web Service using a secure SQLite environment.

Step 3 – Remediation

Incorporate the system into operational pipelines to automate target audits and generate clean, runnable CLI logic.

Image Sources



https://miro.medium.com/1*TmepouA5yUIX3A2R8ZpRfw.jpeg

Source: medium.com



<https://global.discourse-cdn.com/streamlit/original/3X/4/5/4553efed580466554357cc01a9cc064c6d742bf9.jpeg>

Source: discuss.streamlit.io



<https://cdn.thenewstack.io/media/2022/09/c5932e0c-transactions-costs.png>

Source: thenewstack.io